

Activity 15: GitHub Version Control Practice

Arul Santoshi

2024-11-22

Table of contents

1	Introduction	1
2	Analysis 1: Sex and Rank in the US Armed Forces	2
2.1	Interpretation of Frequency Table	2
3	Analysis 2: Popular Baby Names Trends	3
3.1	Interpretation of Baby Names Trends	3
4	Analysis 3: The Box Problem - Maximizing Volume	5
4.1	Analysis of the Box Volume Function	5
5	Data Summary	6
6	Reflection on Course Learning	7
7	Code Appendix	8
7.1	Load Packages	8
7.2	Load Armed Forces Data	8
7.3	Clean Armed Forces Data	8
7.4	Create Group-Level Data	8
7.5	Create Individual-Level Data	10
7.6	Verify Data	10
7.7	Load Baby Names Data	10
7.8	Filter Baby Names Data	10
7.9	Create Baby Names Time Series Visualization	11
7.10	Define Box Volume Function	11
7.11	Create Box Problem Visualization	12

1 Introduction

This portfolio presents three distinct statistical analyses demonstrating data wrangling, visualization, and interpretation skills. The first section examines active-duty personnel in the US Armed

Forces by service branch, sex, and pay grade using data from June 2025. The second section explores naming trends for classic literary names using historical baby name data from 1880 to 2014. The third section solves an optimization problem involving the construction of an open-top box from a rectangular piece of paper.

2 Analysis 1: Sex and Rank in the US Armed Forces

For this analysis, I examine **Enlisted personnel in the Army** to explore whether sex and rank are independent of each other. The Army's enlisted ranks range from E1 (entry-level) through E9 (senior enlisted), providing a comprehensive view of the military hierarchy. Table 1 displays the frequency distribution of male and female Army enlisted personnel across all nine rank levels.

Table 1: Table 1: Two-Way Frequency Table of Enlisted Personnel in the Army by Sex and Rank

Sex	E1	E2	E3	E4	E5	E6	E7	E8	E9	Total
Male	7,429	22,338	43,775	79,234	54,803	49,502	30,264	9,482	2,865	299,692
Female	1,326	4,336	10,229	15,143	10,954	7,363	4,410	1,472	394	55,627
Total	8,755	26,674	54,004	94,377	65,757	56,865	34,674	10,954	3,259	355,319

2.1 Interpretation of Frequency Table

Table 1 reveals a clear pattern suggesting that sex and rank are **not independent** in the Army's enlisted personnel. The table displays the distribution of 299,692 male and 55,627 female enlisted soldiers across nine rank levels (E1 through E9). Several key observations emerge from examining the frequency distributions and their relationship to the marginal totals.

First, females consistently represent a smaller proportion of personnel at every rank level, with an overall representation of approximately 15.7% (55,627 out of 355,319 total enlisted). However, the proportion of females varies noticeably across ranks. At the entry level (E1), females account for 15.1% of personnel (1,326 out of 8,755). This proportion increases slightly in the middle ranks, with females representing 16.3% at E2 (4,336 out of 26,674) and 18.9% at E3 (10,229 out of 54,004). The female representation then decreases as rank increases, dropping to 16.1% at E4, 16.7% at E5, and declining further to approximately 12.1% at E9 (394 out of 3,259).

If sex and rank were independent, we would expect the proportion of females to remain constant across all rank levels, matching the overall 15.7% representation. The observed variation (ranging from 12.1% at E9 to 18.9% at E3) suggests that the distribution of males and females across ranks differs from what would be expected under independence. The declining representation of females at higher enlisted ranks (E7 through E9) is particularly noteworthy, as these senior enlisted positions require longer service tenure and represent leadership roles within the military structure.

The marginal totals show that lower ranks (E1 through E4) contain the majority of personnel, with E4 being the largest group at 94,377 soldiers. As ranks increase, the total number of personnel decreases substantially, with only 3,259 soldiers at the E9 level. This pyramidal structure is expected in military organizations. However, the fact that female representation does not follow the same proportional distribution across this hierarchy provides evidence that sex and rank are associated rather than independent variables within the Army's enlisted personnel structure.

3 Analysis 2: Popular Baby Names Trends

For this analysis, I examine naming trends for a collection of classic literary names: Elizabeth, Jane, Emma, Charlotte, and Margaret. I selected these names because they represent enduring characters from English literature, from Jane Austen's heroines (Elizabeth Bennet, Emma Woodhouse, Jane Bennet) to Charlotte Brontë and Margaret Mitchell. These names have appeared consistently throughout American history, making them ideal for exploring long-term naming trends and cultural shifts in name popularity. Figure 1 displays the time series of these five names from 1880 to 2014, showing how literary influence and cultural preferences have shaped naming patterns across generations.

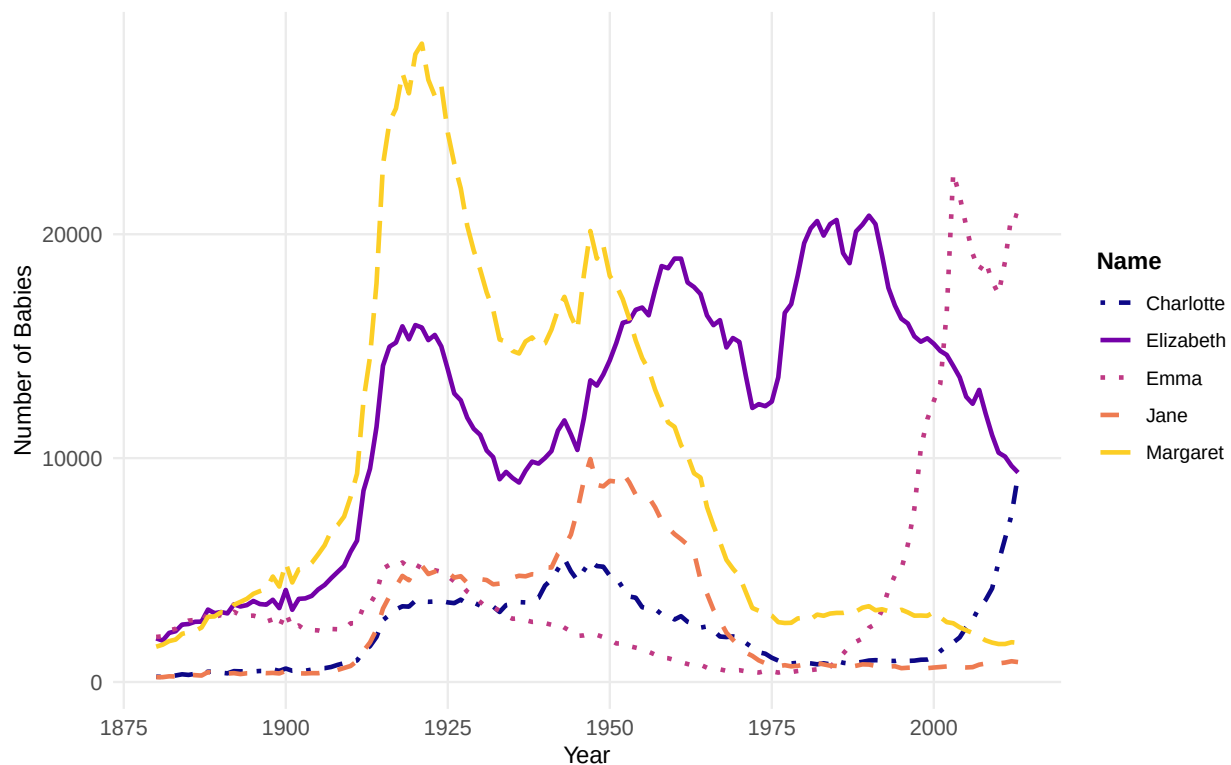


Figure 1: Figure 1: Popularity Trends of Classic Literary Names in the United States (1880-2014)

3.1 Interpretation of Baby Names Trends

Figure 1 reveals distinct popularity patterns for these five classic literary names across 134 years of American naming history. The visualization uses both color and line type to distinguish between names, ensuring accessibility for color-blind readers while maintaining visual clarity. The y-axis displays the total number of babies given each name per year, while the x-axis spans from 1880 to 2014.

Elizabeth demonstrates the most dramatic trajectory, rising steadily from the early 1900s to peak around 1960 with over 25,000 babies receiving this name annually. This peak coincides with the reign of Queen Elizabeth II and the post-World War II era's preference for traditional, regal names.

Following this zenith, Elizabeth’s popularity declined gradually but remained substantial, never dropping below 10,000 annual occurrences even in recent decades.

Margaret shows a contrasting pattern, beginning with strong popularity in the early 1900s when it ranked among the most common names, with nearly 20,000 babies named Margaret in some years. However, this name experienced a consistent decline throughout the twentieth century, falling to fewer than 2,500 occurrences by 2014. This decline reflects a broader cultural shift away from Victorian-era names.

Jane follows a bell-curve pattern, with moderate popularity in the early decades, peaking in the 1940s and 1950s at around 12,000 babies per year, then declining steadily to become relatively uncommon by 2014. The mid-century peak aligns with the popularity of actress Jane Wyman and the broader cultural prominence of simple, elegant names during that era.

Emma presents the most striking recent trend. For most of the twentieth century, Emma remained relatively unpopular, dipping below 2,000 annual occurrences in the 1970s and 1980s. However, beginning in the 1990s, Emma experienced a remarkable resurgence, climbing to become one of America’s most popular names by 2014, rivaling Elizabeth’s historical peak. This revival likely reflects renewed interest in Jane Austen’s works and a broader trend toward vintage name revival.

Charlotte maintained relatively low and stable popularity throughout most of the period, typically ranging between 2,000 and 5,000 babies per year. However, the line shows a notable uptick beginning in the 2000s, suggesting Charlotte is participating in the same vintage name revival trend as Emma. The combined use of the viridis color palette and distinct line types (solid, dashed, dotted, dotdash, and longdash) ensures that these trends remain distinguishable even for readers with color vision deficiencies.

4 Analysis 3: The Box Problem - Maximizing Volume

The Box Problem involves starting with a rectangular piece of paper and creating an open-top box by cutting out equal-sized squares from each of the four corners and folding up the sides. For this analysis, I use a piece of paper measuring 36 inches by 48 inches. The fundamental question is: what side length for the square cutouts will produce the box with the largest possible volume?

When we cut out squares of side length x from each corner, the resulting box has dimensions: length = $(48 - 2x)$, width = $(36 - 2x)$, and height = x . The volume is therefore $V(x) = x(48 - 2x)(36 - 2x)$. The value of x must be positive and cannot exceed 18 inches (half the shorter dimension), since cutting larger squares would eliminate the material needed for the box sides. Figure 2 displays the volume function across all valid values of x , allowing us to identify the optimal square size without using calculus.

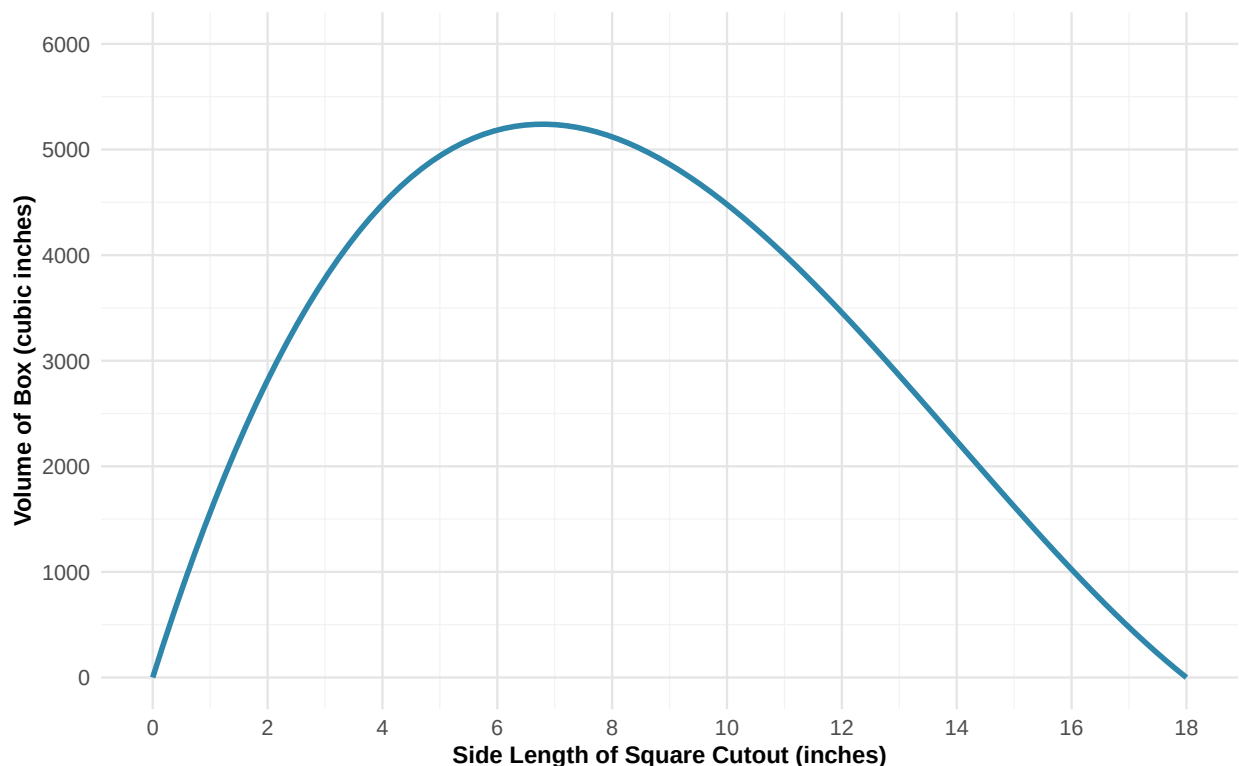


Figure 2: Figure 2: Volume of Open-Top Box as a Function of Square Cutout Side Length

4.1 Analysis of the Box Volume Function

Figure 2 displays the relationship between the side length of the square cutouts (x) and the resulting volume of the open-top box. The graph shows a smooth, curved function that begins at zero volume when $x = 0$ inches (no box is created when no squares are cut), rises to a clear maximum, and returns to zero as x approaches 18 inches (when the cutouts become so large that the box collapses).

The most important feature of this visualization is the distinct peak in the curve, which occurs at approximately $x = 6$ inches. At this cutout size, the volume function reaches its maximum

value of approximately 5,184 cubic inches. This can be verified by examining the curve: to the left of $x = 6$ inches, the volume increases as the cutout size grows; to the right of $x = 6$ inches, the volume decreases as the cutouts become too large. The symmetrical nature of the parabola-like curve confirms that there is a single optimal solution.

To answer the original questions precisely: the **maximum volume is approximately 5,184 cubic inches**, achieved when the **side length of the square cutouts is approximately 6 inches**. At this optimal cutout size, the resulting box has dimensions of approximately 6 inches in height, 36 inches in length (48 minus 2 times 6), and 24 inches in width (36 minus 2 times 6). These dimensions produce the largest possible volume for an open-top box created from a 36-inch by 48-inch rectangular piece of paper.

The graph clearly shows why smaller or larger cutouts produce suboptimal results. Cutouts smaller than 6 inches create boxes that are too shallow, failing to use the available paper height efficiently. Cutouts larger than 6 inches create taller boxes but sacrifice too much base area, as the length and width decrease rapidly. The optimal solution at $x = 6$ inches represents the perfect balance between the box's height and its base dimensions, maximizing the product of all three measurements.

5 Data Summary

Table 2: Total Personnel by Category

Category	Total_Personnel
Commissioned Officer	216,079
Enlisted	1,041,489
Warrant Officer	20,594

6 Reflection on Course Learning

Throughout this course, I have developed substantial skills in data wrangling, visualization, and statistical communication that have fundamentally changed how I approach data analysis. The most significant learning has been understanding that effective data analysis is not just about running code correctly, but about making principled decisions at every step of the workflow.

In data wrangling, I learned the importance of tidy data principles and thoughtful data transformation. The Armed Forces project demonstrated this clearly when I had to make decisions about how to structure the data. Initially, I might have simply kept the data in its original wide format, but I learned to recognize when pivoting to long format makes subsequent analysis more intuitive and flexible. The process of separating combined variables (like “Branch_Sex” into separate columns) taught me that clean, well-structured data makes analysis easier and less error-prone. Similarly, working with the BabyNames dataset reinforced the value of grouping and summarizing data appropriately. By combining counts across sex for the literary names analysis, I was able to focus on overall naming trends rather than getting lost in unnecessary detail.

Visualization design has been perhaps the most transformative aspect of the course. Before this class, I viewed graphs primarily as ways to display results. Now I understand that visualizations are communication tools that require careful design decisions. The baby names time series taught me that accessibility matters. Using both color and line type ensures that readers with color vision deficiencies can still interpret the graph, a consideration I never made before. The box problem visualization showed me the power of the grammar of graphics approach in ggplot2. Using `stat_function()` to plot a mathematical function directly, rather than computing points manually, produces cleaner code and more precise visualizations. I also learned that effective visualizations need appropriate scales. My initial box plot had a truncated y-axis that cut off the maximum, obscuring the most important feature of the graph. Expanding the axis to show the full curve made the optimal solution immediately apparent.

Writing about data has improved significantly through practice with narrative interpretation. Early in the course, I tended to simply describe what the visualization showed without connecting it to context or answering the underlying question. The Armed Forces frequency table analysis taught me to go beyond reporting numbers to explaining what patterns mean. Rather than just stating that female representation varies across ranks, I learned to calculate percentages, compare them to what we would expect under independence, and discuss the implications for understanding military personnel structure. The box problem taught me to explicitly answer the original questions (maximum volume and optimal cutout size) while also explaining the reasoning visible in the graph.

Perhaps most importantly, I have internalized the principle that code should be written for humans to read, not just for computers to execute. Using explicit argument names (like `data = literary_names` rather than just `literary_names` as the first argument) makes code self-documenting. Writing meaningful variable names (`group_level_df` instead of `df1`) and adding comments that explain the purpose of code sections helps both my future self and other readers understand my analysis. The discipline of organizing code into logical sections in the appendix, with clear labels for each chunk, has made my workflow more systematic and reproducible.

These skills connect directly to my goals in data science. The ability to take messy real-world data, transform it appropriately, visualize it effectively, and communicate findings clearly is exactly what professional data work requires. I have moved from seeing R as a tool for calculation to understanding it as a comprehensive environment for reproducible statistical analysis and communication.

7 Code Appendix

All code used in this analysis is displayed below. The data wrangling process includes loading packages, reading from Google Sheets, cleaning data, creating group-level and individual-level data frames, and verification.

7.1 Load Packages

```
# Load required packages for data manipulation and visualization
library(package = tidyverse)
library(package = googlesheets4)
library(package = knitr)
```

7.2 Load Armed Forces Data

```
# Disable authentication for public Google Sheets access
gs4_deauth()

# Read the Armed Forces data from Google Sheets
# Skip first 2 rows (title and branch headers)
# Read all columns as character to preserve formatting
armed_forces_raw <- read_sheet(
  ss = "https://docs.google.com/spreadsheets/d/19xQnI1cBh6Jkw7eP8YQuuicMlVDF7Gr-nXCb5qbwb_E/ed.
  skip = 2,
  col_types = "c"
)
```

7.3 Clean Armed Forces Data

```
# Clean the raw data by renaming and filtering
armed_forces_clean <- armed_forces_raw %>%
  # Rename first column to PayGrade for clarity
  rename(PayGrade = `Pay Grade`) %>%
  # Remove rows where PayGrade is missing
  filter(!is.na(PayGrade)) %>%
  # Remove total/summary rows and source note
  filter(!str_detect(string = PayGrade, pattern = "Total|Source"))
```

7.4 Create Group-Level Data


```

# Create group-level data frame with counts by PayGrade, Branch, and Sex
group_level_df <- armed_forces_clean %>%
# Select relevant columns, excluding "Total" columns for each branch
# Column names have ...N suffix due to duplicate "Male"/"Female" names
select(
  PayGrade,
  Army_Male = Male...2,
  Army_Female = Female...3,
  Navy_Male = Male...5,
  Navy_Female = Female...6,
  MarineCorps_Male = Male...8,
  MarineCorps_Female = Female...9,
  AirForce_Male = Male...11,
  AirForce_Female = Female...12,
  SpaceForce_Male = Male...14,
  SpaceForce_Female = Female...15
) %>%
# Reshape data from wide to long format
pivot_longer(
  cols = -PayGrade,
  names_to = "Branch_Sex",
  values_to = "Count"
) %>%
# Separate Branch_Sex column into two columns
separate(
  col = Branch_Sex,
  into = c("Branch", "Sex"),
  sep = "_"
) %>%
# Clean and convert Count column to numeric
mutate(
  # Remove comma separators from numbers
  Count = str_remove_all(string = Count, pattern = ","),
  # Replace "N/A*" text with actual NA values
  Count = na_if(x = Count, y = "N/A*"),
  # Convert character counts to numeric
  Count = as.numeric(Count),
  # Replace remaining NA values with 0
  Count = replace_na(data = Count, replace = 0)
) %>%
# Add Category column based on PayGrade prefix
mutate(
  Category = case_when(
    str_starts(string = PayGrade, pattern = "E") ~ "Enlisted",
    str_starts(string = PayGrade, pattern = "W") ~ "Warrant Officer",
    str_starts(string = PayGrade, pattern = "O") ~ "Commissioned Officer",
    .default = "Other"
  )
)

```

```

    )
  ) %>%
  # Add Rank column by extracting numeric portion from PayGrade
  mutate(Rank = as.integer(str_extract(string = PayGrade, pattern = "\\d+"))) %>%
  # Remove rows with zero count
  filter(Count > 0)

```

7.5 Create Individual-Level Data

```

# Create individual-level data frame where each row represents one person
individual_level_df <- group_level_df %>%
  # Expand rows based on Count (create Count number of rows for each observation)
  uncount(weights = Count) %>%
  # Add unique identifier for each individual
  mutate(ID = row_number())

```

7.6 Verify Data

```

# Verify that total counts match between group and individual level data
verification_result <- sum(group_level_df$Count) == nrow(individual_level_df)

# Display verification result
cat("Data verification - counts match:", verification_result, "\n")
cat("Total in group_level_df:", format(sum(group_level_df$Count), big.mark = ","), "\n")
cat("Rows in individual_level_df:", format(nrow(individual_level_df), big.mark = ","), "\n")

```

7.7 Load Baby Names Data

```

# Load the dcData package for baby names data
library(package = dcData)

# Load the BabyNames dataset
data("BabyNames")

```

7.8 Filter Baby Names Data

```

# Filter to selected literary names, combining counts across sex
literary_names <- BabyNames %>%
  # Filter to only the chosen names

```

```

filter(name %in% c("Elizabeth", "Jane", "Emma", "Charlotte", "Margaret")) %>%
# Group by name and year (combining male and female counts)
group_by(name, year) %>%
# Sum counts across sex
summarise(total_count = sum(count), .groups = "drop")

```

7.9 Create Baby Names Time Series Visualization

```

# Create time series visualization
ggplot(data = literary_names,
       mapping = aes(x = year,
                     y = total_count,
                     color = name,
                     linetype = name)) +
# Add line geometry
geom_line(linewidth = 1) +
# Set color-blind friendly palette
scale_color_viridis_d(option = "plasma",
                      end = 0.9,
                      name = "Name") +
# Use different line types to distinguish names beyond color
scale_linetype_manual(values = c("Elizabeth" = "solid",
                                "Jane" = "dashed",
                                "Emma" = "dotted",
                                "Charlotte" = "dotdash",
                                "Margaret" = "longdash"),
                      name = "Name") +
# Set clear axis labels
labs(x = "Year",
     y = "Number of Babies",
     title = NULL) +
# Use clean theme
theme_minimal() +
# Customize theme elements
theme(legend.position = "right",
      legend.title = element_text(face = "bold"),
      axis.title = element_text(size = 11),
      axis.text = element_text(size = 10),
      panel.grid.minor = element_blank())

```

7.10 Define Box Volume Function

```

# Define the volume function for a 36" x 48" piece of paper
# x represents the side length of the square cutouts in inches
box_volume <- function(x) {
  # Volume = height * length * width
  # height = x
  # length = 48 - 2*x (original 48" minus two cutouts)
  # width = 36 - 2*x (original 36" minus two cutouts)
  volume <- x * (48 - 2 * x) * (36 - 2 * x)
  return(volume)
}

```

7.11 Create Box Problem Visualization

```

# Create visualization of the box volume function
ggplot(data = data.frame(x = c(0, 18))) +
  # Plot the volume function using stat_function
  stat_function(fun = box_volume,
               linewidth = 1.2,
               color = "#2E86AB") +
  # Set clear axis labels
  labs(x = "Side Length of Square Cutout (inches)",
       y = "Volume of Box (cubic inches)",
       title = NULL) +
  # Set axis limits with expanded y-axis to show full curve
  scale_x_continuous(limits = c(0, 18),
                    breaks = seq(from = 0, to = 18, by = 2)) +
  scale_y_continuous(limits = c(0, 6000),
                    breaks = seq(from = 0, to = 6000, by = 1000)) +
  # Use clean theme
  theme_minimal() +
  # Customize theme elements
  theme(axis.title = element_text(size = 11, face = "bold"),
        axis.text = element_text(size = 10),
        panel.grid.major = element_line(color = "gray90"),
        panel.grid.minor = element_line(color = "gray95"))

```